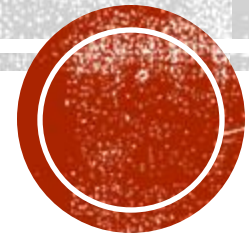


CLOUD COMPUTING

Chandara Chea

chandara.chea@cadt.edu.kh



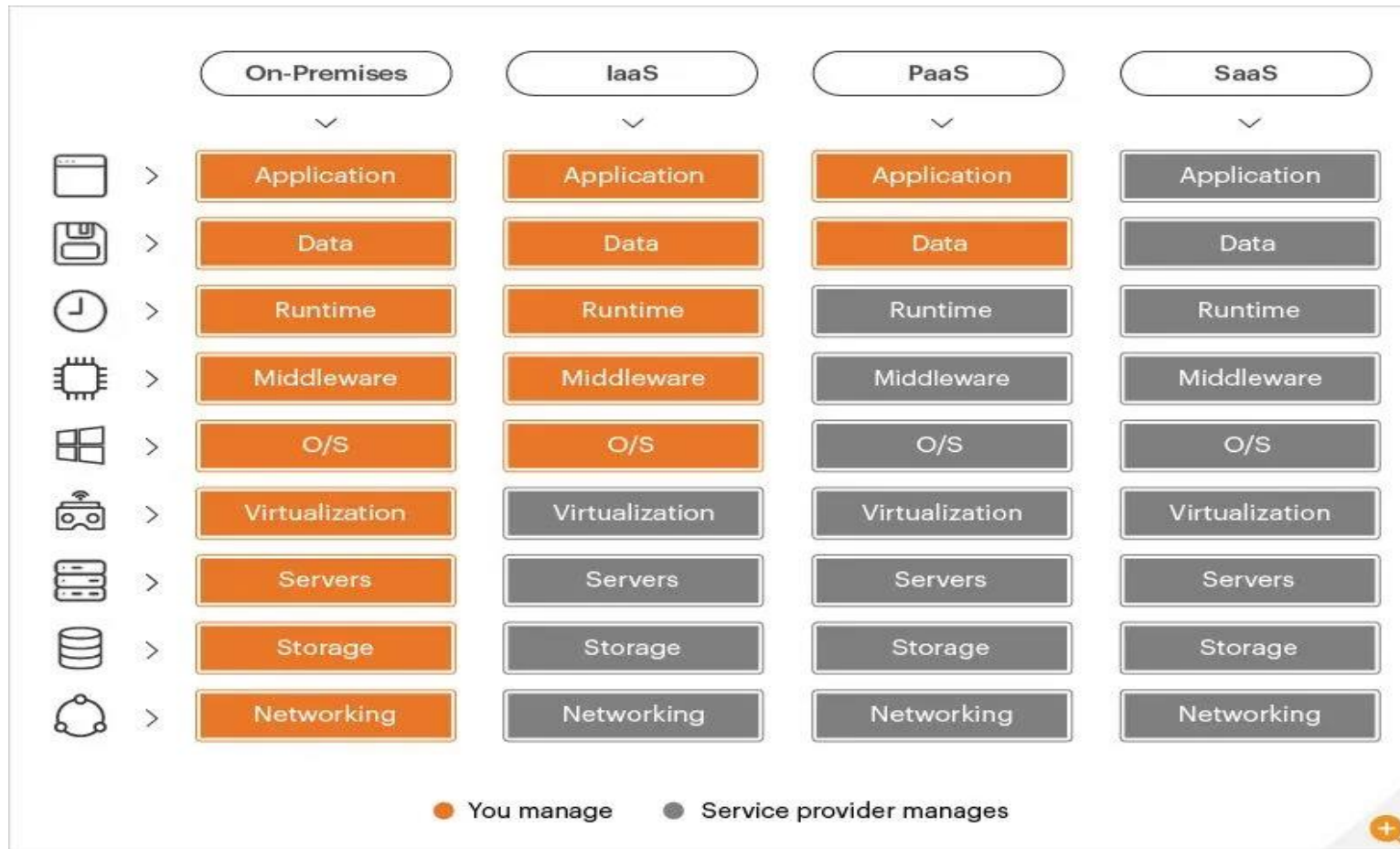
WEEK 5 — CLOUD SECURITY

- **Learning Objectives:**

- Understand cloud service delivery model
- Understand cloud shared responsibility model
- Understand Identity and access management
- Understand Authentication and authorization
- Understand data protection in the cloud



1. CLOUD SERVICE DELIVERY MODEL



- **IaaS:** Infrastructure as a Service
- **PaaS:** Platform as a Service
- **SaaS:** Software as a Service



2. KEY ASPECT CLOUD SECURITY

- **Data protection:** Keeping data safe and available through encryption, backup, and disaster recovery.
- **Access control and identity management:** Controlling who accesses resources and what they can do (authentication, authorization, identity management practices)
- **Network security:** Ensuring the network is secure through segmentation, traffic filtering/monitoring, and defenses against attacks like DoS.
- **Compliance and regulatory requirements:** Adhering to industry regulations (HIPAA, PCI DSS, GDPR).
- **Incident response:** Having a plan to detect, respond to, and recover from security incidents



3. CLOUD SECURITY THREAT & CHALLENGES

- **Data breaches and unauthorized access**
 - Unauthorized access to sensitive data stored in the cloud.
 - Often caused by weak access controls, misconfigured storage (e.g., public S3 buckets), or vulnerabilities in applications.
- **Insecure APIs and interfaces**
 - APIs are the gateway to cloud services, and insecure ones can be exploited to gain unauthorized access.
- **Misconfiguration and human error**
 - Incorrect settings in cloud resources (e.g., open ports, public data).
 - One of the most common causes of cloud-related breaches.
- **Insider threats**
 - Malicious or careless actions by employees, contractors, or third parties who have access to cloud systems.



3. CLOUD SECURITY THREAT & CHALLENGES

- **Account Hijacking**

- Attackers gaining access to cloud accounts (e.g., admin accounts) using phishing, credential stuffing, or weak passwords.

- **Denial of Service (DoS/DDoS) Attacks**

- Overloading cloud services to make them unavailable to legitimate users.

- **Insider Threats**

- Malicious or careless actions by employees, contractors, or third parties who have access to cloud systems.

- **Insufficient Identity and Access Management (IAM)**

- Weak or overly permissive IAM policies can give attackers excessive access to resources.



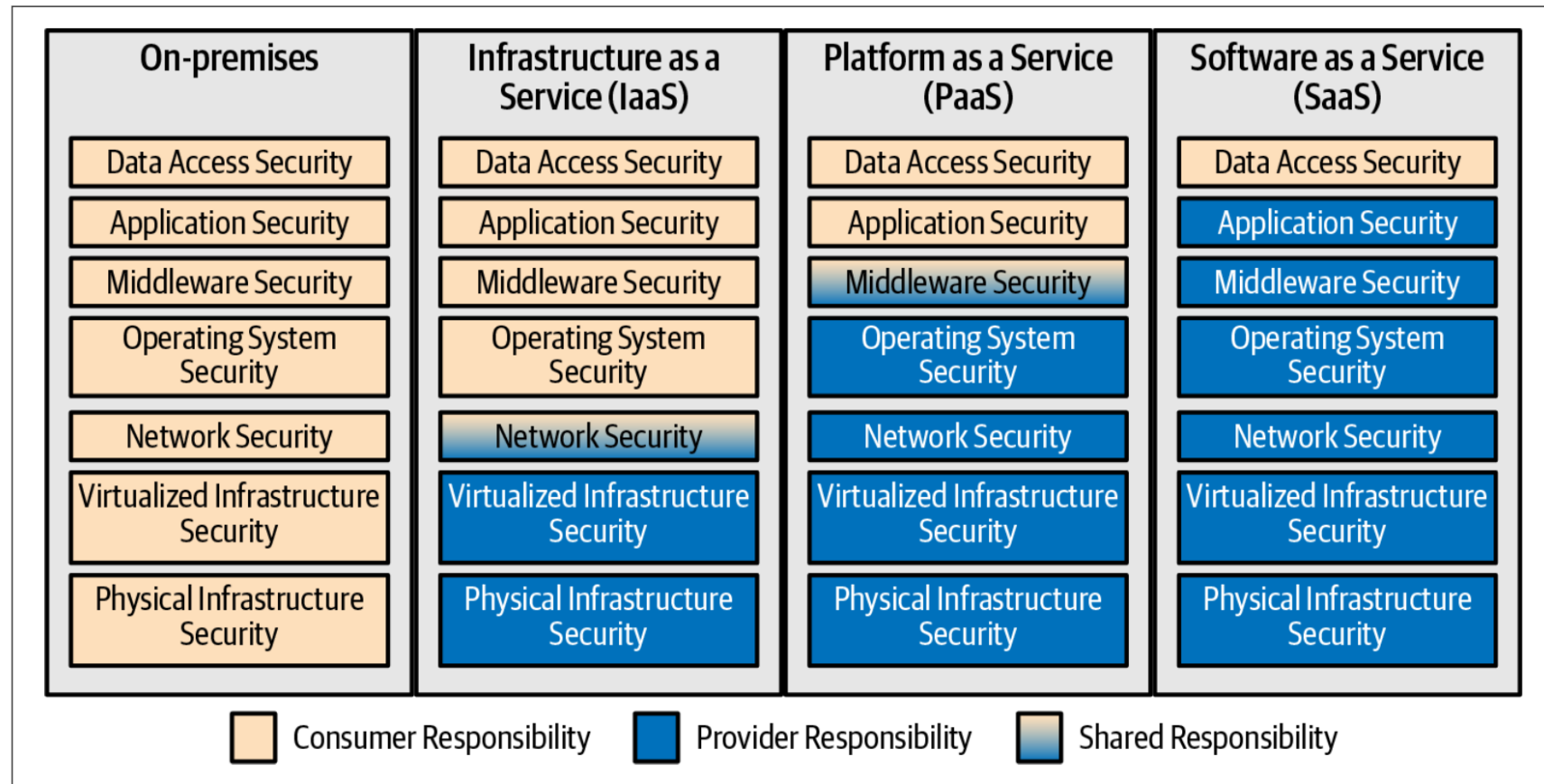
4. SECURE THE CLOUD ENVIRONMENT

- Securing a cloud environment involves a combination of technology, processes, and people. Here are the **best practices and recommended solutions** to build and maintain a secure cloud infrastructure:
 - **Understand the Shared Responsibility Model**
 - **Implement Strong Identity and Access Management (IAM)**
 - **Tokenize and encrypt data**

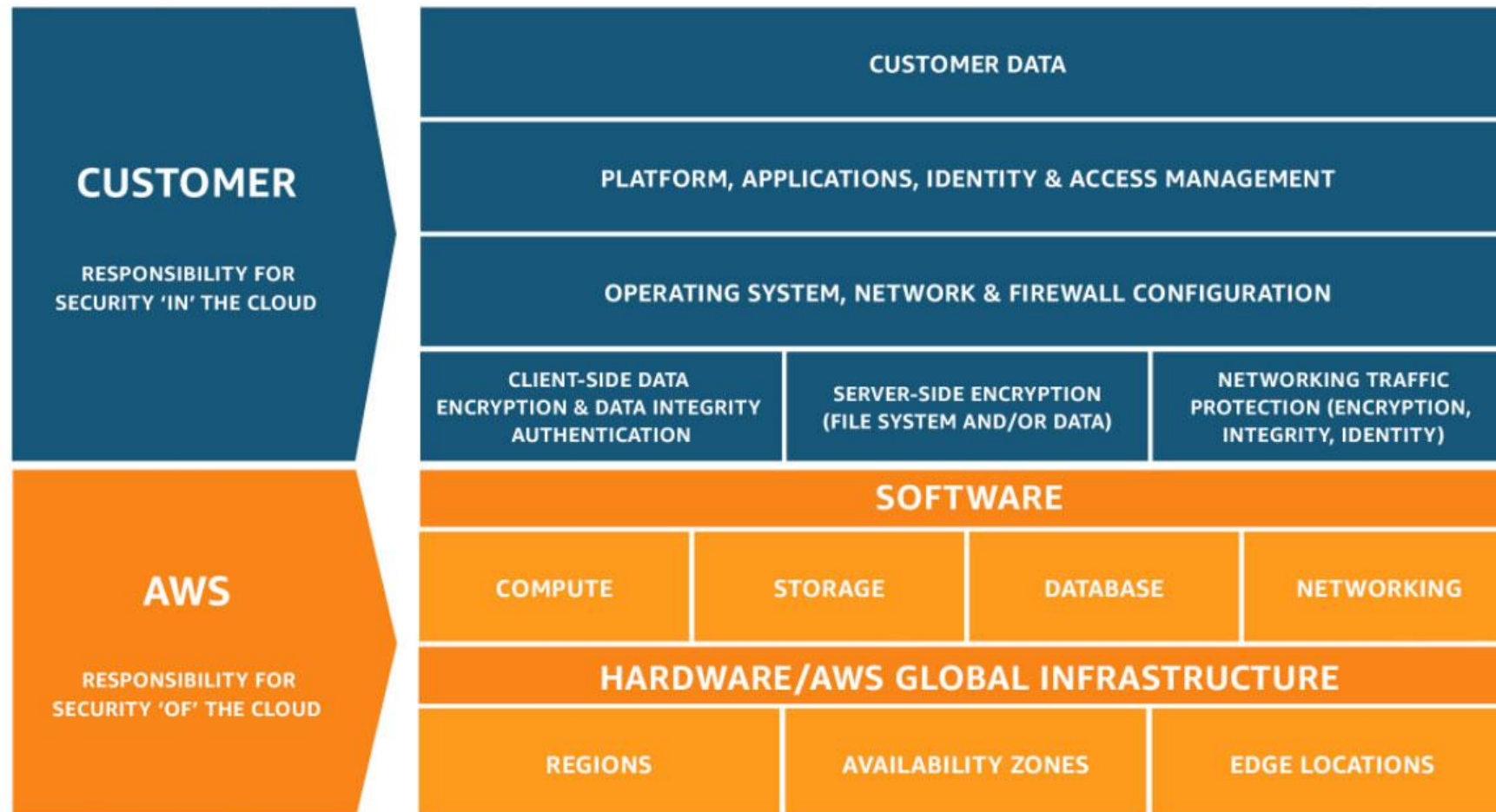


4.1. CLOUD SHARED RESPONSIBILITY MODEL

- “What aspects of security am I responsible for?”



4.1. AWS SHARED RESPONSIBILITY MODEL



4.1. CLOUD SHARED RESPONSIBILITY MODEL

- The cloud provider has complete responsibility for physical infrastructure security—which often involves controls beyond what many companies can reasonably do on-premises, such as biometric access with anti-tailgating measures, security guards, slab-to-slab barriers, and similar controls to keep unauthorized personnel out of the physical facilities.
- Operating system security is usually straightforward: it's your responsibility if you're using IaaS, and it's the provider's responsibility if you're purchasing platform or software services.
- Middleware, in this context, is a generic name for software such as databases, application servers, or queuing systems. They're in the middle between the operating system and the application—not used directly by end users but used to develop solutions for end users. If you're using a PaaS, middleware security is often a shared responsibility.
- Data access security is almost always your responsibility as a customer. If you incorrectly tell your cloud provider to allow access to specific data, such as granting incorrect object storage permissions, middleware permissions, or SaaS permissions, there's really not much the provider can do other than try to detect the problem and warn you.

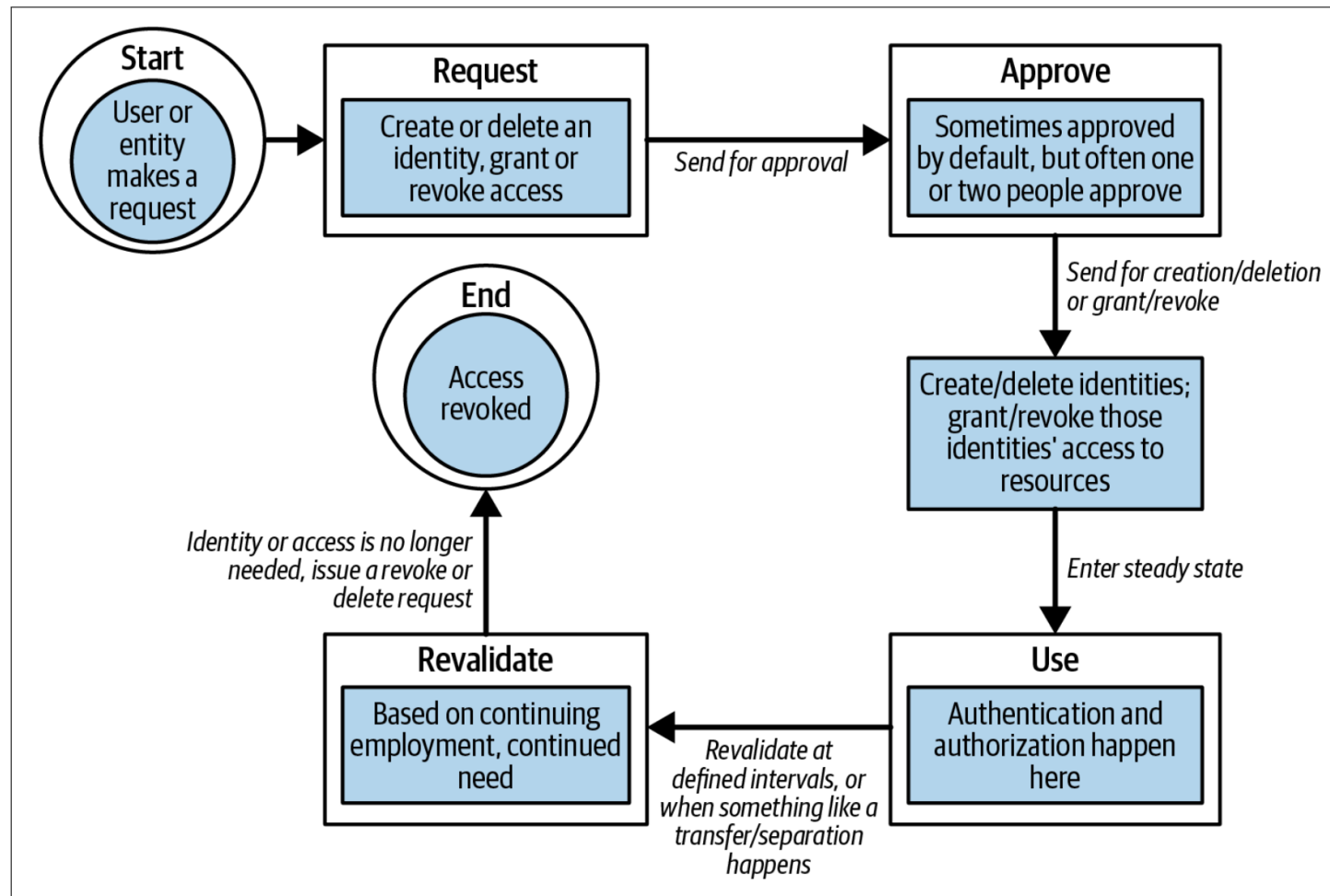


4.2. IDENTITY AND ACCESS MANAGEMENT

- Identity and access management (IAM) is perhaps the most important set of security controls. In breaches involving web applications, lost or stolen credentials have been attackers' most-used tool for several years running. If attackers have valid credentials to log in to your system, all of the patches and firewalls in the world won't keep them out.
- An *identity* is how a person (or automation) is represented in the system. The process of verifying that the entity making a request is really the owner of the identity is called *authentication* (often abbreviated as “authn”).
- *Access management* is about allowing identities to perform the tasks they need to perform (and, in a least privilege environment, *only* the tasks they need to perform). The process of checking what privileges an identity should have is called *authorization* (often abbreviated as “authz”).



4.2. LIFE CYCLE OF IDENTITY AND ACCESS



4.2. AUTHENTICATION METHODS

- Passwords and passphrases
- PINs
- SMS text messages to a mobile device
- One-time passcodes, or OTP
- Push notifications to a mobile device
- Fingerprint readers, face readers, and retina readers



4.2. SHARED ID, FEDERATED IDENTITY, SINGLE SIGN ON (SSO)

- Shared IDs are identities for which more than one person has the password or other credentials, such as the built-in *root* or *Administrator* account on a system. Where possible, every user or tool should have its own ID that's not used by anyone or anything else.
- ***Federated identity*** is a concept, not a specific technology. It means that you may have identities on two different systems, and the administrators of those systems can both agree to use technologies that link those identities together so that you don't have to manually create separate accounts on each system. From your perspective as a user, you have only a single identity.
- Single sign-on (SSO) is a set of technology implementations that rely upon the concept of federated identity. E.g. Google sign-on



4.2. AUTHORIZATION

- Once you've completed the authentication phase and you know who your users are, it's time to make sure they are limited to performing only the actions they are supposed to perform. Some examples of authorization include granting permission to access an application at all, to access an application with write access, to access a portion of the network, or to access the cloud console.
- The most important concepts to remember for authorization are *least privilege* and *separation of duties*.
- Least privilege means that your users, systems, or tools should be able to access only what they need to do their jobs, and no more. In practice, this usually means that you have a "deny by default" policy in place, so that unless you specifically authorize something, it's not allowed.
- Separation (or segregation) of usually translates more generally into making sure that no one person can completely undermine the security of the entire environment. For example, someone with the ability to make changes on systems should not also have the ability to alter the logs from those systems, or the responsibility for reviewing the logs from those systems.



4.3. PROTECTING DATA IN THE CLOUD

- **Tokenization** is a way to protect sensitive data like **credit card numbers** by replacing them with a fake version called a **token**.
- The token looks similar to the real data (for example, still 16 digits long), so systems that expect a number don't need to change.
- The real data is stored securely in one place, called a token service, and only that service can link the token back to the real data.
- Tokenization can be used by itself or together with encryption for even stronger protection.



4.3. PROTECTING DATA IN THE CLOUD

- **Encryption**

- **Data at Rest:** This refers to data stored in cloud storage (e.g., databases, file storage). Cloud providers often automatically encrypt data at rest using encryption algorithms, which ensures that even if an unauthorized party gains access to the storage system, they cannot read the data.
- **Data in Transit:** This refers to data being transmitted between systems (e.g., from your device to the cloud server). Encryption of data in transit (usually through SSL/TLS protocols) ensures that attackers cannot intercept or tamper with data while it's being transferred.



4.4. KEY MANAGEMENT

- Key Management is the process of handling cryptographic keys used to protect data — including creating, storing, rotating, and revoking them securely.
- In the cloud and security world, it's a critical part of data encryption and access control.
- A cryptographic key is a secret code used in encryption and decryption. Think of it as the password that locks and unlocks your data.
- Why Is Key Management Important?
 - Data Protection: Even encrypted data can be exposed if keys are mismanaged.
 - Compliance: Regulations like GDPR, HIPAA, PCI-DSS require strong key control.
 - Access Control: Prevents unauthorized users or systems from accessing sensitive data.



4.4. KEY MANAGEMENT

- There are various approaches:
 - Managed Key Management:
 - The cloud provider manages and stores the keys. While this offers convenience, it also means that the provider has access to the keys, which could be a concern for some organizations.
 - Customer-Managed Keys:
 - The customer create and manage the encryption keys within the cloud provider's Key Management Service (KMS).
 - The customer control key rotation, access policy and key deletion
 - BYOK - Bring Your Own Key:
 - You generate the encryption key outside the cloud, using your own hardware or software tools, and then upload it into the cloud provider's KMS.
 - In case of having strict compliance, legal, or regulatory requirements and needing full control over your key material.

